Early-Game Advantage and Match Outcomes in League of Legends

Name(s): Gabrielle Despaigne

Website Link: https://gdespaigne.github.io/early-advantage-modeling/

In [1]:
```
import pandas as pd
import numpy as np
from pathlib import Path

import plotly.express as px
pd.options.plotting.backend = 'plotly'

# from dsc259r_utils import * # Feel free to uncomment and use this.
```

In [2]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler, QuantileTransformer
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score
from sklearn.impute import SimpleImputer
import plotly.graph_objects as go
from sklearn.metrics import confusion_matrix
```

Step 1: Introduction

Quick Stats About the Dataset

The League of Legends dataset contains detailed statistics from professional matches. Each game is represented by multiple rows, including player-level statistics and team-level summary rows. The dataset includes 165 columns covering match metadata (league, patch, split, date), performance metrics (kills, gold, objectives), and match outcome (result).

The raw dataset contains 12 rows per game:

10 player-level rows (5 per team)

2 team-level summary rows

At this stage, no filtering has been performed. This initial inspection helps identify how the dataset is structured before any cleaning or transformation.

In [3]:
```
## loading in csv
df = pd.read_csv("/Users/gabrielledespaigne/Documents/GitHub/dsc259r-final/data/2022_LoL_esports_match_data_from_OraclesElixir.csv")
df.head()
```

```
/var/folders/ct/62w77r6505s3mm314sbfrrh00000gn/T/ipykernel_49952/3099605488.py:2: DtypeWarning: Columns (2) have mixed types. Specify dtype option on import or set low_memory=False.
  df = pd.read_csv("/Users/gabrielledespaigne/Documents/GitHub/dsc259r-final/data/2022_LoL_esports_match_data_from_OraclesElixir.csv")
```

Out[3]:

	gameid	datacompleteness	url	league	year	split	playoffs	date	game	patch	...	opp_csat25	golddiffat25	xpdiffat25	csdiffat25	killsat25	assistsat25	deathsat25	opp_killsat25	opp_assistsat25	opp_deathsat25
0	ESPORTSTMNT01_2690210	complete	NaN	LCKC	2022	Spring	0	2022-01-10 07:44:08	1	12.01	...	203.0	605.0	-525.0	9.0	0.0	1.0	1.0	0.0	2.0	0.0
1	ESPORTSTMNT01_2690210	complete	NaN	LCKC	2022	Spring	0	2022-01-10 07:44:08	1	12.01	...	163.0	421.0	-903.0	-28.0	2.0	4.0	2.0	1.0	5.0	1.0
2	ESPORTSTMNT01_2690210	complete	NaN	LCKC	2022	Spring	0	2022-01-10 07:44:08	1	12.01	...	187.0	-149.0	-224.0	-5.0	1.0	3.0	0.0	3.0	4.0	3.0
3	ESPORTSTMNT01_2690210	complete	NaN	LCKC	2022	Spring	0	2022-01-10 07:44:08	1	12.01	...	284.0	-1288.0	-2005.0	-85.0	2.0	1.0	2.0	3.0	4.0	0.0
4	ESPORTSTMNT01_2690210	complete	NaN	LCKC	2022	Spring	0	2022-01-10 07:44:08	1	12.01	...	27.0	499.0	-314.0	12.0	1.0	3.0	2.0	0.0	7.0	2.0

5 rows × 165 columns

In [4]:

```
## dataset before any filtering is done
display(df.shape)
display(df.columns)
```

```
(148980, 165)
Index(['gameid', 'datacompleteness', 'url', 'league', 'year', 'split',
       'playoffs', 'date', 'game', 'patch',
       ...
       'opp_csat25', 'golddiffat25', 'xpdiffat25', 'csdiffat25', 'killsat25',
       'assistsat25', 'deathsat25', 'opp_killsat25', 'opp_assistsat25',
       'opp_deathsat25'],
      dtype='object', length=165)
```

Initial Thoughts

From the initial inspection, several important characteristics are evident:

- The dataset includes both player-level and team-level rows.
- There are numerous early-game metrics (e.g., killsat10, golddiffat10).
- Several columns indicate early objective control (e.g., firstdragon, firstherald, firsttower).
- Some columns contain missing values, suggesting structural changes across patches.

Brainstorming

Based on the structure of the dataset, several potential research questions arise:

- Does securing first blood increase a team's probability of winning?
- Are early gold advantages at 10 minutes predictive of match outcome?
- Do certain patches produce more snowball-heavy games?
- Is early objective control more predictive of victory than early kills?
- Does the relationship between early-game advantage and winning differ by league?

Proposed Research Question

After reviewing the available early-game metrics, I focus on the following research question:

Is early objective control a stronger indicator of match outcome than early kills in professional League of Legends matches?

This question is compelling because it contrasts macro-level strategic control (objectives) with combat-based advantage (kills). Understanding which type of early-game advantage better predicts victory provides insight into competitive strategy and match dynamics.

Importantly, this question can be addressed through both hypothesis testing and predictive modeling, aligning with the later stages of the project.

Step 2: Data Cleaning and Exploratory Data Analysis

Data Cleaning

The raw dataset contains both player-level and team-level rows (12 rows per game). Since this analysis focuses on match outcomes at the team level, I restrict the dataset to rows where position == "team", resulting in exactly two rows per game.

In [5]:
```
df_team = df[df["position"] == "team"].copy()

df_team.shape
```

Out[5]:
```
(24830, 165)
```

In [6]:
```
df_team.groupby("gameid").size().value_counts()
```

Out[6]:
```
2    12415
Name: count, dtype: int64
```

This results in 24,830 team-level observations, with exactly two rows per game.

In [7]:
```
bool_cols = ["firstdragon", "firstherald", "firsttower", "firstblood"]

for col in bool_cols:
    df_team[col] = df_team[col].fillna(0).astype(int)
```

In [8]:
```
objective_cols = ["firstdragon", "firstherald", "firsttower"]

df_team["early_objective_control"] = (
    df_team[objective_cols].sum(axis=1) >= 1
).astype(int)
```

To capture macro-level early advantage, I created a new binary variable, early_objective_control, defined as securing at least one of firstdragon, firstherald, or firsttower.

Univariate Analysis

The dataset is approximately balanced between wins and losses, as expected since each game contributes one win and one loss.

In [9]:
```
df_team["result_label"] = df_team["result"].map({
    0: "Loss",
    1: "Win"
})

fig1 = px.histogram(
    df_team,
    x="result_label",
    color="result_label",
    title="League of Legends 2022 - Distribution of Match Outcomes",
    labels={
        "result_label": "Match Outcome",
        "count": "Number of Matches"
    },
    color_discrete_map={
        "Loss": "red",
        "Win": "blue"
    }
)

fig1.update_layout(
    yaxis_title="Number of Matches",
```

```
    template="plotly_white"
)

fig1.show()
```

In [10]:
```
df_team["objective_label"] = df_team["early_objective_control"].map({
    0: "No",
    1: "Yes"
})

fig2 = px.histogram(
    df_team,
    x="objective_label",
    color="objective_label",
    title="League of Legends 2022 - Distribution of Early Objective Control",
    labels={
        "objective_label": "Early Objective Control",
        "count": "Number of Matches"
    },
    color_discrete_map={
        "No": "red",
        "Yes": "blue"
    }
)

fig2.update_layout(
    yaxis_title="Number of Matches",
    template="plotly_white"
)

fig2.show()
```

Bi-variate Analysis

In [11]:
```
## yes/no early objective results
[c for c in df_team.columns if "result" in c.lower() or "win" in c.lower()]
```

Out[11]:
```
['result', 'result_label']
```

In [12]:
```
## yes - 0, no - 1
df_team["result"].value_counts()
```

Out[12]:
```
result
0    12415
1    12415
Name: count, dtype: int64
```

Teams with early objective control have a substantially higher win rate (55.8%) compared to teams without (32.8%).

In [13]:
```
## finding early objective terms
[c for c in df_team.columns if "first" in c.lower()]
```

Out[13]:
```
['firstPick',
 'firstblood',
 'firstbloodkill',
 'firstbloodassist',
 'firstbloodvictim',
 'firstdragon',
 'firstherald',
 'firstbaron',
 'firsttower',
 'firstmidtower',
 'firsttothreetowers']
```

In [14]:
```
## wanting to see what items / "firsts" named contain dragon, herald, or tower
[c for c in df_team.columns if "dragon" in c.lower() or "herald" in c.lower() or "tower" in c.lower()]
```

Out[14]:
```
['firstdragon',
 'dragons',
 'opp_dragons',
 'dragons (type unknown)',
 'firstherald',
 'heralds',
 'opp_heralds',
 'firsttower',
 'towers',
 'opp_towers',
 'firstmidtower',
 'firsttothreetowers',
 'damagetotowers']
```

Note: Going to use firstdragon, firstherald, firsttower for early game indicators because others like dragons are total dragons for the whole game. Firstbaron is usually midgame and firsttothreetowers can happen midgame.

So by these standards, objective control = 1 if team secured at least one of [firstdragon, firstherald, firsttower] otherwise it's 0.

In [15]:
```
df_team[["firstdragon", "firstherald", "firsttower"]].head()
```

Out[15]:

	firstdragon	firstherald	firsttower
10	0	1	1
11	1	0	0
22	0	1	0
23	1	0	1
34	0	0	1

In [16]:
```
df_team[["firstdragon", "firstherald", "firsttower"]].sum()
```

Out[16]:
```
firstdragon    11315
firstherald    10519
firsttower     11329
dtype: int64
```

In [17]:
```
## wins with early objective control, raw numbers
objective_cols = ["firstdragon", "firstherald", "firsttower"]

df_team["early_objective_control"] = (
    df_team[objective_cols].sum(axis=1) >= 1
).astype(int)

df_team["early_objective_control"].value_counts()
```

Out[17]:
```
early_objective_control
1    18589
0     6241
Name: count, dtype: int64
```

In [18]:
```
df_team.groupby("early_objective_control")["result"].mean()
```

Out[18]:
```
early_objective_control
0    0.328473
1    0.557588
Name: result, dtype: float64
```

In [19]:
```
win_rates = (
    df_team.groupby("early_objective_control")["result"]
    .mean()
    .reset_index()
)
```

```python
win_rates["objective_label"] = win_rates["early_objective_control"].map({
    0: "No",
    1: "Yes"
})

fig4 = px.bar(
    win_rates,
    x="objective_label",
    y="result",
    color="objective_label",
    title="League of Legends 2022 — Win Rate by Early Objective Control",
    labels={
        "objective_label": "Early Objective Control",
        "result": "Win Rate"
    },
    color_discrete_map={
        "No": "red",
        "Yes": "blue"
    }
)

fig4.update_layout(
    yaxis_title="Win Rate",
    template="plotly_white",
    showlegend=False
)

fig4.update_yaxes(tickformat=".0%")

fig4.show()
```

In [20]:

```python
win_by_fb = (
    df_team.groupby("firstblood")["result"]
    .mean()
    .reset_index()
)

win_by_fb["firstblood_label"] = win_by_fb["firstblood"].map({
    0: "No First Blood",
    1: "First Blood Secured"
})

fig3 = px.bar(
    win_by_fb,
    x="firstblood_label",
    y="result",
    color="firstblood_label",
    title="League of Legends 2022 — Win Rate by First Blood",
    labels={
        "firstblood_label": "First Blood",
        "result": "Win Rate"
    },
    color_discrete_map={
        "No First Blood": "red",
        "First Blood Secured": "blue"
    }
)

fig3.update_layout(
    yaxis_title="Win Rate",
    template="plotly_white",
    showlegend=False
)

fig3.update_yaxes(tickformat=".0%")
```

```
fig3.show()
```

This allows comparison between combat-based early advantage (first blood) and objective-based advantage.

In [21]:
```
## objective effect
obj_rates = df_team.groupby("early_objective_control")["result"].mean()
obj_effect = obj_rates[1] - obj_rates[0]

## first blood effect
fb_rates = df_team.groupby("firstblood")["result"].mean()
fb_effect = fb_rates[1] - fb_rates[0]

obj_effect, fb_effect
```

Out[21]:
```
(np.float64(0.22911481963259428), np.float64(0.22021810893981475))
```

In [22]:
```
effect_df = pd.DataFrame({
    "Metric": ["Early Objective Control", "First Blood"],
    "Win Rate Increase": [obj_effect, fb_effect]
})

fig5 = px.bar(
    effect_df,
    x="Metric",
    y="Win Rate Increase",
    color="Metric",
    title="League of Legends 2022 - Comparison of Early-Game Advantage Impact on Win Rate",
    labels={"Win Rate Increase": "Increase in Win Rate"},
    color_discrete_map={
        "Early Objective Control": "blue",
        "First Blood": "red"
    }
)

fig5.update_layout(
    template="plotly_white",
    showlegend=False
)

fig5.update_yaxes(tickformat=".0%")

fig5.show()
```

In [23]:
```
comparison_df = pd.DataFrame({
    "Metric": ["Objective", "Objective", "First Blood", "First Blood"],
    "Condition": ["No Advantage", "Advantage Secured",
                  "No Advantage", "Advantage Secured"],
    "Win Rate": [
        obj_rates[0],
        obj_rates[1],
        fb_rates[0],
        fb_rates[1]
    ]
})
```

In [24]:
```
comparison_df["Metric"] = comparison_df["Metric"].replace({
    "Objective": "Early Objective Control"
})

fig6 = px.bar(
    comparison_df,
    x="Metric",
    y="Win Rate",
    color="Condition",
    barmode="group",
```

```
    title="League of Legends 2022 - Win Rates by Type of Early Advantage",
    color_discrete_map={
        "No Advantage": "red",
        "Advantage Secured": "blue"
    }
)

fig6.update_layout(
    template="plotly_white"
)

fig6.update_yaxes(tickformat=".0%")

fig6.show()
```

While both early objective control and first blood substantially increase win probability, their effect sizes are remarkably similar. This suggests that both macro-level control and early combat advantage play comparably important roles in determining match outcomes.

Interesting Aggregates

This table shows how the relationship between early objective control and win rate varies across patches. In most patches, teams with early objective control maintain a consistent advantage.

In [25]:
```
## note 0-no, 1-yes
agg_table = (
    df_team.groupby(["patch", "early_objective_control"])["result"]
    .mean()
    .unstack()
)

agg_table.head()
```

Out[25]:

early_objective_control	0	1
patch		
12.01	0.236559	0.558566
12.02	0.363328	0.559322
12.03	0.246429	0.550141
12.04	0.402490	0.556901
12.05	0.310606	0.551921

Generate Markdowns for Website

In [26]:
```
## decided to limit the columns shown in head otherwise it becomes too wide
## only relevant columns that I actually looked at are included
cols_preview = [
    "result",
    "side",
    "firstblood",
    "firstdragon",
    "firstherald",
    "firsttower",
    "void_grubs",
    "early_objective_control",
    "golddiffat10",
    "killsat10"
]

print(df_team[cols_preview].head(10).to_markdown(index=False))
```

result	side	firstblood	firstdragon	firstherald	firsttower	void_grubs	early_objective_control	golddiffat10	killsat10
0	Blue	1	0	1	1	nan	1	1523	3
1	Red	0	1	0	0	nan	1	-1523	0
0	Blue	0	0	1	0	nan	1	-1619	1
1	Red	1	1	0	1	nan	1	1619	3
1	Blue	0	0	0	1	0	1	nan	nan
0	Red	1	1	0	0	0	1	nan	nan
1	Blue	0	1	0	1	nan	1	-103	0
0	Red	1	0	1	0	nan	1	103	1
1	Blue	1	0	0	1	0	1	nan	nan
0	Red	0	1	0	0	0	1	nan	nan

In [27]:
```
print(agg_table.to_markdown())
```

patch	0	1
12.01	0.236559	0.558566
12.02	0.363328	0.559322
12.03	0.246429	0.550141
12.04	0.40249	0.556901
12.05	0.310606	0.551921
12.06	0.288136	0.540193
12.07	0.206897	0.562963
12.08	0.174603	0.568562
12.09	0.210526	0.556507
12.1	0.243697	0.554856
12.11	0.327652	0.567407
12.12	0.346041	0.5525
12.13	0.380303	0.559488
12.14	0.307947	0.565463
12.15	0.195122	0.566138
12.16	0.361789	0.57173
12.17	0.166667	0.547619
12.18	0.190751	0.55991
12.19	0.185714	0.567901
12.2	0.290323	0.542763
12.21	0.225806	0.550296
12.23	0.5	0.5

In [28]:
```
fig1.write_html("assets/win_distribution.html", include_plotlyjs="cdn")
```

In [29]:
```
fig5.write_html("assets/objective_winrate.html", include_plotlyjs="cdn")
```

Step 3: Assessment of Missingness

In [30]:
```
## a look at columns to explore for missingness
df_team.isna().sum().sort_values(ascending=False).head(15)
```

Out[30]:
```
atakhans                  24830
playername                24830
opp_atakhans              24830
firstbloodkill            24830
firstbloodassist          24830
firstbloodvictim          24830
damageshare               24830
champion                  24830
playerid                  24830
earnedgoldshare           24830
total cs                  24830
void_grubs                23240
opp_void_grubs            23240
monsterkillsownjungle     20958
monsterkillsenemyjungle   20958
dtype: int64
```

In [31]:
```python
missing = df_team.isna().sum()
missing = missing[(missing > 0) & (missing < len(df_team))]
missing.sort_values(ascending=False).head(15)
```

Out[31]:
```
void_grubs                 23240
opp_void_grubs             23240
monsterkillsenemyjungle    20958
monsterkillsownjungle      20958
url                        20946
dragons (type unknown)     20410
split                       6856
opp_deathsat25              5074
killsat25                   5074
goldat25                    5074
opp_goldat25                5074
opp_xpat25                  5074
opp_csat25                  5074
golddiffat25                5074
xpdiffat25                  5074
dtype: int64
```

In [32]:
```python
## does missingness of void_grubs depend on patch?
## it missing in most does suggest it wasnt added until after this year
df_team["void_grubs_missing"] = df_team["void_grubs"].isna().astype(int)
df_team["void_grubs_missing"].value_counts()
```

Out[32]:
```
void_grubs_missing
1    23240
0     1590
Name: count, dtype: int64
```

In [33]:
```python
## is grubs missing from every patch in the year? this suggests no, that it was recorded for some patches
df_team.groupby("patch")["void_grubs_missing"].mean()
```

Out[33]:
```
patch
12.01    0.863103
12.02    0.874615
12.03    1.000000
12.04    0.912844
12.05    0.958435
12.06    1.000000
12.07    1.000000
12.08    1.000000
12.09    1.000000
12.10    0.908148
12.11    0.919063
12.12    0.942580
12.13    0.934608
12.14    0.902357
12.15    0.858696
12.16    1.000000
12.17    1.000000
12.18    1.000000
12.19    1.000000
12.20    1.000000
12.21    1.000000
12.23    1.000000
Name: void_grubs_missing, dtype: float64
```

Column Selection

The column void_grubs contains substantial missingness (23,240 missing values out of 24,830 team-level rows). Because this missingness is non-trivial and appears structurally meaningful, it is selected for further analysis.

To analyze missingness, I create a missingness indicator:

In [34]:
```python
df_team["void_grubs_missing"] = df_team["void_grubs"].isna().astype(int)
```

```
df_team["void_grubs_missing"].value_counts()
```

Out[34]:
```
void_grubs_missing
1    23240
0     1590
Name: count, dtype: int64
```

This confirms that the majority of observations have missing values for this column.

NMAR Analysis

To determine whether the missingness is likely Not Missing At Random (NMAR), reason must be applied to how the data was generated.

Void Grubs were introduced in later League of Legends patches (post 2024) and did not exist in earlier patches. Therefore, in matches played before their introduction, this column would naturally be missing. Since patch version is observed in the dataset, and missingness can plausibly be explained by patch version, the missingness of void_grubs is consistent with Missing At Random (MAR), not NMAR.

There is no evidence that missingness depends on the value of void_grubs itself. As such, the data are unlikely to be NMAR.

Missingness Dependency Tests

Dependency on Patches

From exploratory analysis, missingness rates varied across patch versions:

Some patches exhibit 100% missingness, while others show slightly lower rates.

To simplify the test, patches were grouped into:

- High-missingness patches (rate = 1.0)
- Partially-missing patches (rate < 1.0)

In [35]:
```
df_team.groupby("patch")["void_grubs_missing"].mean()
```

Out[35]:
```
patch
12.01    0.863103
12.02    0.874615
12.03    1.000000
12.04    0.912844
12.05    0.958435
12.06    1.000000
12.07    1.000000
12.08    1.000000
12.09    1.000000
12.10    0.908148
12.11    0.919063
12.12    0.942580
12.13    0.934608
12.14    0.902357
12.15    0.858696
12.16    1.000000
12.17    1.000000
12.18    1.000000
12.19    1.000000
12.20    1.000000
12.21    1.000000
12.23    1.000000
Name: void_grubs_missing, dtype: float64
```

In [36]:
```
## grouping variable to start permutation testing
df_team["high_missing_patch"] = (
    df_team["patch"].isin(
        df_team.groupby("patch")["void_grubs_missing"]
        .mean()
        .loc[lambda x: x == 1.0]
```

```
        .index
    )
).astype(int)

df_team["high_missing_patch"].value_counts()
```

Out[36]:
```
high_missing_patch
0    18574
1     6256
Name: count, dtype: int64
```

In [37]:
```
## observed diff
obs_diff = (
    df_team.groupby("high_missing_patch")["void_grubs_missing"]
    .mean()
    .diff()
    .iloc[-1]
)

obs_diff
```

Out[37]:
```
np.float64(0.08560353181867131)
```

Observed difference ≈ 0.0856

In [38]:
```
n_perms = 5000
perm_diffs = []

for _ in range(n_perms):
    shuffled = df_team["void_grubs_missing"].sample(frac=1).values
    temp = df_team.copy()
    temp["shuffled"] = shuffled

    diff = (
        temp.groupby("high_missing_patch")["shuffled"]
        .mean()
        .diff()
        .iloc[-1]
    )

    perm_diffs.append(diff)

perm_diffs = np.array(perm_diffs)

p_value = np.mean(perm_diffs >= obs_diff)

obs_diff, p_value
```

Out[38]:
```
(np.float64(0.08560353181867131), np.float64(0.0))
```

p-value ≈ 0.0

The permutation test yields a p-value approximately equal to 0, indicating strong evidence that missingness of void_grubs depends on patch version. This supports the earlier reasoning that missingness is structurally tied to patch changes.

Independence from Side

Next, we test whether missingness depends on team side (Blue vs Red). There is no theoretical reason that side selection would influence whether Void Grubs existed in a patch.

Null Hypothesis:

Missingness of void_grubs is independent of side.

Alternative Hypothesis:

Missingness of void_grubs depends on side.

In [39]:
```
obs_side_diff = (
    df_team.groupby("side")["void_grubs_missing"]
    .mean()
    .diff()
    .iloc[-1]
)

obs_side_diff
```

Out[39]:
```
np.float64(0.0)
```

In [40]:
```
n_perms = 5000
perm_diffs_side = []

for _ in range(n_perms):
    shuffled = df_team["void_grubs_missing"].sample(frac=1, replace=False).values

    temp = df_team.copy()
    temp["shuffled_missing"] = shuffled

    diff = (
        temp.groupby("side")["shuffled_missing"]
        .mean()
        .diff()
        .iloc[-1]
    )

    perm_diffs_side.append(diff)

perm_diffs_side = np.array(perm_diffs_side)

p_value_side = np.mean(np.abs(perm_diffs_side) >= np.abs(obs_side_diff))

obs_side_diff, p_value_side
```

Out[40]:
```
(np.float64(0.0), np.float64(1.0))
```

The permutation test yields a large p-value, indicating no evidence that missingness of void_grubs depends on team side. This aligns with expectations, as side selection is unrelated to patch-level structural changes.

In [41]:
```
fig_side = px.histogram(
    perm_diffs_side,
    nbins=40,
    title="Does Void Grubs Missingness Depend on Team Side?",
    labels={"value": "Difference in Missingness Rate, Blue vs Red Team"}
)

fig_side.data[0].name = "Permutation Distribution"

fig_side.add_vline(
    x=obs_side_diff,
    line_color="red",
    line_width=3,
    annotation_text="Observed Difference",
    annotation_position="top"
)

## highlighting extreme values used for the p-value
extreme = [x for x in perm_diffs_side if abs(x) >= abs(obs_side_diff)]

fig_side.add_histogram(
    x=extreme,
    nbinsx=40,
    marker_color="rgba(255,0,0,0.4)",
    name="Values Used for p-value"
)

fig_side.update_layout(
```

```
    template="plotly_white",
    xaxis_title="Difference in Missingness Rate, Blue vs Red Team",
    yaxis_title="Number of Permutations",
    legend_title="Distribution Type"
)

fig_side.show()
```

Export Plot to Markdown

In [42]:
```
fig_side.write_html("assets/missingness_permutation.html", include_plotlyjs=True)
```

Step 4: Hypothesis Testing

Question 1:

Does securing early objective control increase the probability of winning a match?

Hypotheses

Null Hypothesis (H_0):

The probability of winning is the same for teams with early objective control and teams without early objective control.

Alternative Hypothesis (H_1):

Teams with early objective control have a higher probability of winning than teams without early objective control.

This is a one-sided test because we are specifically testing whether early objective control increases win probability.

Test Statistic

The test statistic is the difference in win proportions:

Win rate (early objective = 1) – Win rate (early objective = 0)

This measures how much early objective control shifts the probability of winning.

In [43]:
```
obs_diff = (
    df_team.groupby("early_objective_control")["result"]
    .mean()
    .diff()
    .iloc[-1]
)

obs_diff
```

Out[43]:
```
np.float64(0.22911481963259428)
```

The observed difference in win rates is approximately 22.9 percentage points.

In [44]:
```
n_perms = 5000
perm_diffs = []

for _ in range(n_perms):
    shuffled = df_team["result"].sample(frac=1, replace=False).values

    temp = df_team.copy()
    temp["shuffled_result"] = shuffled
```

```
    diff = (
        temp.groupby("early_objective_control")["shuffled_result"]
        .mean()
        .diff()
        .iloc[-1]
    )

    perm_diffs.append(diff)

perm_diffs = np.array(perm_diffs)

p_value = np.mean(perm_diffs >= obs_diff)

obs_diff, p_value
```

Out[44]:
```
(np.float64(0.22911481963259428), np.float64(0.0))
```

Under null hypothesis, match outcomes are independent of early objective control.

Summary

The permutation test yields a p-value approximately equal to 0, indicating that the observed difference in win rates is extremely unlikely under the null hypothesis. Therefore, we reject the null hypothesis and conclude that early objective control is strongly associated with match outcome.

This provides statistical evidence that early objective control significantly increases a team's probability of winning.

Question 2:

Does securing first blood increase the probability of winning?

Hypotheses

Null Hypothesis (H_0):

Win probability is independent of securing first blood.

Alternative Hypothesis (H_1):

Teams that secure first blood have a higher probability of winning.

Test Statistic

Difference in win proportions:

Win rate (firstblood = 1) − Win rate (firstblood = 0)

In [45]:
```
obs_fb_diff = (
    df_team.groupby("firstblood")["result"]
    .mean()
    .diff()
    .iloc[-1]
)

obs_fb_diff
```

Out[45]:
```
np.float64(0.22021810893981475)
```

In [46]:
```
n_perms = 5000
perm_fb_diffs = []

for _ in range(n_perms):
    shuffled = df_team["result"].sample(frac=1).values
```

```
    temp = df_team.copy()
    temp["shuffled_result"] = shuffled

    diff = (
        temp.groupby("firstblood")["shuffled_result"]
        .mean()
        .diff()
        .iloc[-1]
    )

    perm_fb_diffs.append(diff)

perm_fb_diffs = np.array(perm_fb_diffs)

p_value_fb = np.mean(perm_fb_diffs >= obs_fb_diff)

obs_fb_diff, p_value_fb
```

Out[46]:
```
(np.float64(0.22021810893981475), np.float64(0.0))
```

Summary

The permutation test yields a p-value approximately equal to 0, indicating that the observed difference in win rates is extremely unlikely under the null hypothesis. Therefore, we reject the null hypothesis and conclude that first blood is strongly associated with match outcome.

This provides statistical evidence that first blood significantly increases a team's probability of winning.

Step 5: Framing a Prediction Problem

Prediction Problem - Match Outcome

The prediction problem is to forecast match outcome (result) using only early game features from professional League of Legends matches.

The response variable is result, a binary variable indicating whether a team won (1) or lost (0). This is a binary classification problem.

The features used for prediction will be restricted to early game indicators, including:

- Early objective control (defined as first dragon, herald, or tower)
- First blood

Restricting the model to early game features ensures that predictions are made using only information available in the early phase of the match. This aligns with the broader research theme of understanding how early game advantages translate into final outcomes.

This problem is meaningful because it evaluates whether early strategic advantages provide sufficient information to reliably forecast match success. While hypothesis testing established strong associations, predictive modeling allows us to evaluate how well those early signals generalize to unseen matches.

In addition to evaluating predictive performance, I will examine whether early objective control provides additional predictive signal beyond first blood by comparing model coefficients and performance metrics.

Step 6: Baseline Model

Baseline model

Logistic regression classifier predicting match outcome (result) using two early-game binary indicators: early objective control and first blood.

The baseline is implemented as a single sklearn Pipeline including feature scaling and model training. Performance is evaluated on a held-out test set and with 5-fold cross-validation using accuracy and ROC-AUC to assess generalization.

In [47]:
```
y = df_team["result"]

X_base = df_team[["early_objective_control", "firstblood"]]
```

```
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_base, y, test_size=0.30, random_state=42, stratify=y
)

baseline_clf = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

In [48]:
```
baseline_clf.fit(Xb_train, yb_train)

yb_pred = baseline_clf.predict(Xb_test)
yb_prob = baseline_clf.predict_proba(Xb_test)[:, 1]

base_acc = accuracy_score(yb_test, yb_pred)
base_auc = roc_auc_score(yb_test, yb_prob)

base_acc, base_auc
```

Out[48]:
```
(0.6191435091958652, np.float64(0.6441370324180538))
```

In [49]:
```
base_cv_auc = cross_val_score(baseline_clf, X_base, y, cv=5, scoring="roc_auc")
base_cv_acc = cross_val_score(baseline_clf, X_base, y, cv=5, scoring="accuracy")

np.mean(base_cv_auc), np.std(base_cv_auc), np.mean(base_cv_acc), np.std(base_cv_acc)
```

Out[49]:
```
(np.float64(0.6445718116377026),
 np.float64(0.009207824184918105),
 np.float64(0.6190898107128474),
 np.float64(0.006221757766509259))
```

Step 7: Final Model

Final Model

I aim to improve the baseline logistic regression model from Step 6 by incorporating additional early-game features that capture the magnitude of advantage (such as gold and kills at 10 minutes), and by engineering transformed features to reduce skew/outlier influence.

I will tune model regularization strength and transformation hyperparameters using GridSearchCV, evaluating performance with ROC-AUC to prioritize generalization.

In [50]:
```
y = df_team["result"]

X_final = df_team[["early_objective_control", "firstblood", "golddiffat10", "killsat10", "side"]]

Xf_train, Xf_test, yf_train, yf_test = train_test_split(
    X_final, y, test_size=0.30, random_state=42, stratify=y
)
```

Hyperparameters to tune:

- LogisticRegression C: controls regularization strength and therefore the bias/variance tradeoff and generalization.
- QuantileTransformer n_quantiles applied to golddiffat10: controls how strongly the distribution is reshaped; this may affect separability for a linear model.

I use GridSearchCV with 5-fold cross-validation and ROC-AUC as the main metric to select hyperparameters that generalize well.

In [51]:
```
## Realized there was missingness with golddiffat10 and killsat10 - each have 3,786 missing values.
df_team.isna().sum().sort_values(ascending=False)
```

Out[51]:
```
opp_atakhans          24830
playername            24830
firstbloodkill        24830
firstbloodassist      24830
firstbloodvictim      24830
                      ...
opp_barons                0
firsttower                0
towers                    0
opp_towers                0
high_missing_patch        0
Length: 170, dtype: int64
```

In [52]:
```
gold_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("qt", QuantileTransformer(output_distribution="normal", random_state=42))
])

kills_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

binary_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent"))
])

cat_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess_final = ColumnTransformer(
    transformers=[
        ("gold", gold_pipe, ["golddiffat10"]),
        ("kills", kills_pipe, ["killsat10"]),
        ("bin", binary_pipe, ["early_objective_control", "firstblood"]),
        ("cat", cat_pipe, ["side"]),
    ],
    remainder="drop"
)

final_pipe = Pipeline(steps=[
    ("preprocess", preprocess_final),
    ("model", LogisticRegression(max_iter=2000))
])
```

In [53]:
```
param_grid = {
    "model__C": [0.01, 0.1, 1, 10, 100],
    "preprocess__gold__qt__n_quantiles": [50, 100, 200],
}

grid = GridSearchCV(
    final_pipe,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=5,
    n_jobs=-1,
    refit=True
)

grid.fit(Xf_train, yf_train)

grid.best_params_, grid.best_score_
```

Out[53]:
```
({'model__C': 10, 'preprocess__gold__qt__n_quantiles': 200},
 np.float64(0.7527941362741928))
```

In [54]:
```
best_model = grid.best_estimator_

final_clf = best_model

yf_pred = best_model.predict(Xf_test)
yf_prob = best_model.predict_proba(Xf_test)[:, 1]

final_test_acc = accuracy_score(yf_test, yf_pred)
final_test_auc = roc_auc_score(yf_test, yf_prob)

final_test_acc, final_test_auc
```

Out[54]:
```
(0.6745871929118002, np.float64(0.744767047051954))
```

In [55]:
```
results_table = pd.DataFrame(
    {
        "Baseline": {
            "Test Accuracy": base_acc,
            "Test ROC-AUC": base_auc,
        },
        "Final": {
            "Test Accuracy": final_test_acc,
            "Test ROC-AUC": final_test_auc,
        },
    }
)

print(results_table.to_string())
```

```
              Baseline     Final
Test Accuracy  0.619144  0.674587
Test ROC-AUC   0.644137  0.744767
```

In [56]:
```
cm = confusion_matrix(yf_test, yf_pred)

cm_df = pd.DataFrame(
    cm,
    index=["Actual Loss", "Actual Win"],
    columns=["Predicted Loss", "Predicted Win"]
)

fig_cm = px.imshow(
    cm_df,
    text_auto=True,
    color_continuous_scale="Blues",
    title="2022 League of Legends - Confusion Matrix of Final Model"
)

fig_cm.update_layout(
    template="plotly_white",
    xaxis_title="Predicted Outcome",
    yaxis_title="Actual Outcome"
)

fig_cm.show()
```

In [57]:
```
fig_cm.write_html(
    "assets/confusion_matrix_final_model.html",
    include_plotlyjs="cdn"
)
```

Summary

The final model improves upon the baseline model in both accuracy and ROC-AUC on the held-out test set. Test accuracy increased from 0.619 to 0.675, and ROC-AUC increased from 0.644 to 0.745. This indicates that incorporating magnitude-based early-game features, such as gold difference and kill difference at 10 minutes, provides additional predictive signal beyond binary indicators like early objective control and first blood alone.

Hyperparameters were tuned using GridSearchCV with 5-fold cross-validation and ROC-AUC as the primary metric to prioritize generalization performance. The consistent improvement suggests that the final model better distinguishes winning and losing teams based on early-game dynamics.

Step 8: Fairness Analysis

In [58]:
```
yf_pred = final_clf.predict(Xf_test)

fair_df = Xf_test.copy()
fair_df["true"] = yf_test.values if hasattr(yf_test, "values") else yf_test
fair_df["pred"] = yf_pred

fair_df["group"] = np.where(
    fair_df["early_objective_control"] == 1,
    "Early objective secured",
    "No early objective"
)

fair_df["correct"] = (fair_df["true"] == fair_df["pred"]).astype(int)

acc_by_group = fair_df.groupby("group")["correct"].mean()
acc_by_group
```

Out[58]:
```
group
Early objective secured    0.680029
No early objective         0.658537
Name: correct, dtype: float64
```

In [59]:
```
obs_diff = acc_by_group["Early objective secured"] - acc_by_group["No early objective"]
obs_diff
```

Out[59]:
```
np.float64(0.021492176093790394)
```

Null Hypothesis (H_0):

The model is fair with respect to early objective control. Any difference in test accuracy between teams that secured early objectives and teams that did not is due to random chance.

Alternative Hypothesis (H_1):

The model is unfair with respect to early objective control. The model's test accuracy is different between the two groups.

In [60]:
```
n_perms = 2000
perm_diffs = np.empty(n_perms)

labels = fair_df["group"].values

for i in range(n_perms):
    shuffled = np.random.permutation(labels)

    df_perm = fair_df.copy()
    df_perm["group_perm"] = shuffled

    acc_perm = df_perm.groupby("group_perm")["correct"].mean()
    perm_diffs[i] = acc_perm["Early objective secured"] - acc_perm["No early objective"]

p_value = np.mean(np.abs(perm_diffs) >= abs(obs_diff))
p_value
```

Out[60]:
```
np.float64(0.0875)
```

In [61]:
```
fig_fair = px.histogram(
    perm_diffs,
    nbins=40,
    title="Accuracy Difference by Early Objective Control",
```

```
    labels={"value": "Accuracy Difference, Secured vs Not Secured"}
)

fig_fair.data[0].name = "Permutation Distribution"

fig_fair.add_vline(
    x=obs_diff,
    line_color="red",
    line_width=3,
    annotation_text="Observed Difference",
    annotation_position="top"
)

fig_fair.update_layout(
    template="plotly_white",
    xaxis_title="Accuracy Difference, Secured vs Not Secured",
    yaxis_title="Number of Permutations",
    legend_title="Distribution Type",
    showlegend=True
)

fig_fair.show()
```

In [62]:

```
## export to html
fig_fair.write_html("assets/fairness_permutation.html", include_plotlyjs="cdn")
```

Conclusion

To evaluate whether the model performs differently depending on early-game conditions, I compared prediction accuracy between matches where teams secured early objectives and matches where they did not.

The model achieved an accuracy of 0.680 for teams that secured early objectives and 0.659 for teams that did not, producing an observed difference of 0.0215.

A permutation test was conducted to determine whether this difference could arise due to random chance.

The resulting p-value was 0.0865, which is greater than the 0.05 significance threshold.

Therefore, we fail to reject the null hypothesis and conclude that there is no strong statistical evidence that the model performs differently across these two groups. While the model performs slightly better when early objectives are secured, the difference is small and not statistically significant.